19. Gün — OpenAI Agents SDK ve AsyncIO

19. Gün — OpenAI Agents SDK, Multi-Agent ve AI Sales Outreach Assistant

1. OpenAI Agents SDK

OpenAI Agents SDK, **AI Agent uygulamaları geliştirmek için kullanılan açık kaynaklı, code-first bir agent framework'üdür.**

Sadece OpenAI modellerine bağlı değildir. Uygun bağlantılar sağlandığında farklı modeller de kullanılabilir. Temel yaklaşımı geliştiriciden mümkün olduğunca az kontrol almak ve gereksiz **framework magic** oluşturmamaktır.

SDK içerisinde gördüğümüz temel yapılar:

```
OpenAI Agents SDK
        ↓
Agent
Runner
Tools
Agents as Tools
Handoffs
Guardrails
Tracing
Memory
Streaming
MCP
```

Biz SDK'yı kodda özellikle:

```
from agents import Agent, Runner
```

gibi importlarda görüyoruz.

`Agent()`, `Runner.run()`, `trace()`, `@function_tool`, `SQLiteSession`, `run_streamed()` gibi yapılar SDK tarafından sağlanıyor.

2. AsyncIO

Agent sistemlerinde sürekli dış sistemleri bekliyoruz:

```
Agent
 ↓
LLM API
 ↓
bekleme
```

```
Agent
 ↓
Tool / API
 ↓
bekleme
```

Bu yüzden **AsyncIO** önem kazanıyor.

AsyncIO nedir?

AsyncIO, Python'da asenkron programlama yapmamızı sağlayan yapıdır.

Temel fikir:

```
İş A çalışır
 ↓
A dışarıdan cevap bekler
 ↓
İş B ilerleyebilir
 ↓
A'nın cevabı gelir
 ↓
A devam eder
```

Yani program bekleme süresini daha verimli kullanır.

Bu **gerçek CPU paralelliği değildir**; işlemler concurrent/eşzamanlı şekilde ilerler.

async

Normal fonksiyon:

```
def task():
```

Asenkron fonksiyon:

```
async def task():
```

`async def` ile oluşturulan yapı bir **coroutine**'dir.

Coroutine

Coroutine, çalışması duraklatılıp daha sonra kaldığı yerden devam ettirilebilen asenkron yapıdır.

```
def
 ↓
Normal Function

async def
 ↓
Coroutine
```

await

Bir async işlemin sonucunu beklemek için:

```
result = await task()
```

kullanılır.

`await` sırasında işlem I/O bekliyorsa Event Loop başka uygun async işleri ilerletebilir.

Event Loop

Event Loop, coroutine'lerin çalışma düzenini yönetir.

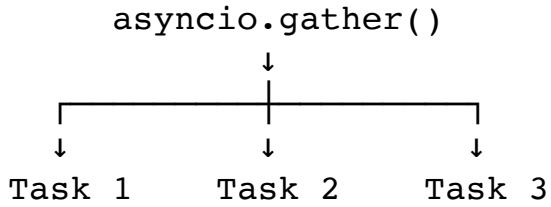
```
Coroutine A
 ↓
await
 ↓
A bekliyor
 ↓
Event Loop
 ↓
Coroutine B
 ↓
A'nın sonucu geldi
 ↓
A devam eder
```

asyncio.gather()

Birden fazla bağımsız async işi birlikte ilerletmek için:

```
await asyncio.gather(
    task1(),
    task2(),
    task3()
)
```

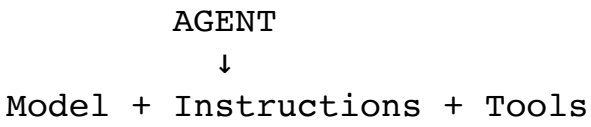
kullanabiliriz.



Bu özellikle LLM, API, HTTP ve database gibi **I/O-bound** işlemlerde değerlidir.

3. Agent Nedir?

Agents SDK'da Agent'ı temel olarak şöyle düşünebiliriz:



Örneğin:

```
Career Agent

Model → LLM
Instructions → "Kariyer analizi yap."
Tools → GitHub Tool
```

Agent oluşturduğumuzda onun **kim olduğunu, görevini ve sahip olduğu araçları** belirliyoruz.

Önemli mühendislik yaklaşımı da şu:

Önce problemi tek Agent ile çözmeye çalış; gerçekten görev ayrımı gerekiyorsa Multi-Agent'a geç.

Çünkü daha fazla Agent otomatik olarak daha iyi sistem anlamına gelmez.

4. Runner ve Agent Loop

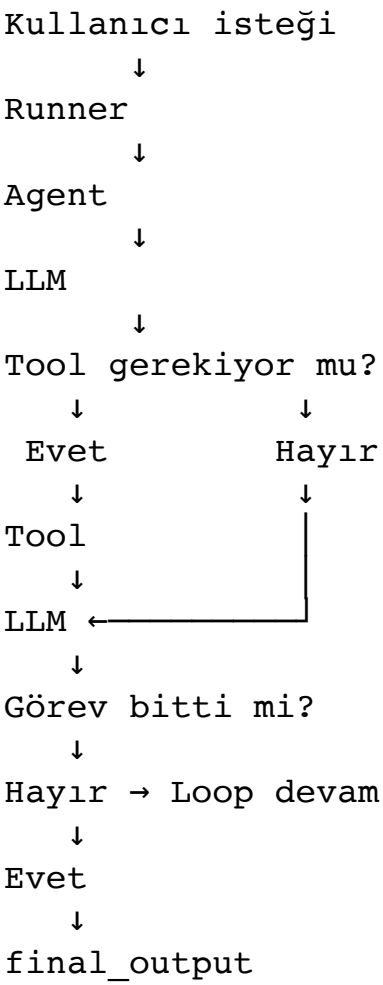
Agent'ı tanımlamak onu çalıştırmak için yeterli değildir.

Bunun için:

```
result = await Runner.run(agent, prompt)
```

kullanıyoruz.

Runner, agent'ın çalışma döngüsünü yürütür:



Yani **Runner, Agent Loop'u çalıştıran yapıdır.**

Son cevaba:

```
result.final_output
```

ile erişebiliriz.

5. Tools

Normal bir Python fonksiyonunu Agent'ın kullanabileceği bir araca dönüştürebiliriz:

```
@function_tool
def push_tool(message: str):
    ...
```

Sonra Agent'a:

```
tools=[push_tool]
```

olarak verebiliriz.

Mantık:

```
Agent
 ↓
Tool Calling
 ↓
Python Function
 ↓
API / Database / Dış Sistem
 ↓
Sonuç
 ↓
Agent
```

Tool'lar sayesinde LLM sadece metin üreten bir yapı olmaktan çıkıp dış dünyada işlem yapabilir.

6. Memory

Agent'ların önceki bağımsız çağrıları otomatik hatırlamasını bekleyemeyiz.

Memory için iki yaklaşım gördük.

Manuel History

```
Önceki konuşma
 ↓
```

```
to_input_list()
 ↓
Yeni Runner çağrısına ekle
```

SQLiteSession

```
session = SQLiteSession("12346")
```

Aynı session sonraki çağrılarda kullanıldığında konuşma geçmişi yönetilebilir.

```
Mesaj 1
 ↓
SQLiteSession
 ↓
Mesaj 2
 ↓
Önceki context kullanılabilir
```

Kısaca:

Memory, geçmiş konuşma bilgisinin sonraki Agent çağrılarında tekrar context olarak kullanılmasını sağlar.

7. Tracing ve Observability

Agent'ın yalnızca son cevabını görmek bazen yeterli değildir.

Şunları da bilmek isteriz:

```
Agent başladı
 ↓
LLM çağrıldı
 ↓
Tool çağrıldı
 ↓
Tool sonucu geldi
 ↓
LLM tekrar çağrıldı
 ↓
Sonuç
```

Buna **Observability** diyoruz.

Tracing ise bunu görmemizi sağlayan mekanizmalardan biridir.

Agents SDK'da tracing özelliğinin framework seviyesinde bulunduğunu da gördük.

8. Streaming

Normal çalışma:

```
LLM
 ↓
cevabın tamamı hazırlanır
 ↓
Kullanıcı görür
```

Streaming:

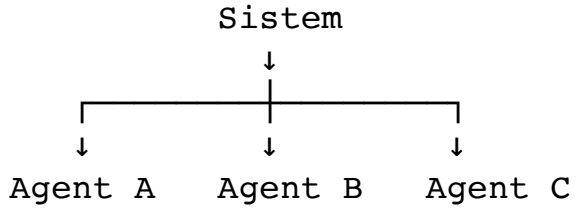
```
LLM
 ↓
parça
 ↓
parça
 ↓
parça
 ↓
Kullanıcı
```

Agents SDK tarafında `Runner.run_streamed()` gibi yapıları gördük.

Daha sonra kendi projemizde de streaming benzeri bir UI akışı oluşturduk; buna birazdan geleceğiz.

9. Multi-Agent

Tek Agent yerine birden fazla uzman Agent birlikte çalışabilir.



Buradaki önemli problem:

Bu Agent'lar hangi sırayla ve nasıl çalışacak?

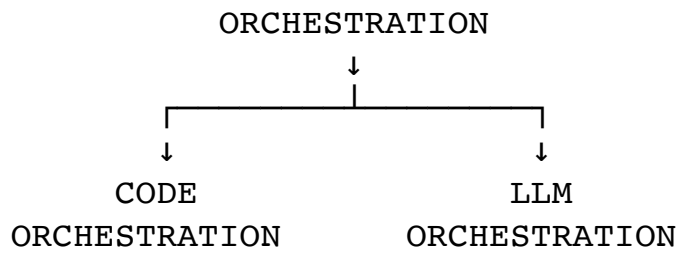
Bu bizi **Orchestration** kavramına götürdü.

10. Agent Orchestration

Orchestration:

Birden fazla Agent'ın hangi sırayla, hangi görevlerle ve nasıl birlikte çalışacağını yönetmektir.

İki temel yaklaşım gördük:



Code Orchestration

Akışı Python belirler:

```
Python
 ↓
Agent 1
 ↓
Agent 2
 ↓
Agent 3
```

Avantajları:

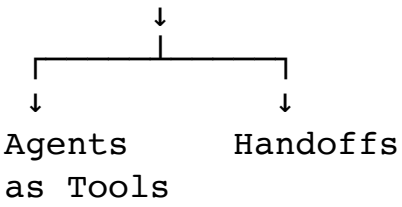
- Daha kontrollü
- Daha öngörülebilir
- Debug etmesi daha kolay
- Akışı önceden belli sistemlere uygun

LLM Orchestration

Hangi Agent'ın çalışacağı konusunda LLM daha fazla karar verir.

İki yöntem gördük:

```
LLM Orchestration
```



Agents as Tools

Bir Agent başka Agent'ı tool gibi kullanır:

```
Agent A
 ↓
Agent B'yi çağır
 ↓
Agent B sonucu
 ↓
Agent A devam eder
```

Kontrol ana Agent'a geri gelir.

Handoff

Görev tamamen başka Agent'a devredilir:

```
Agent A
 ↓
HANDOFF
 ↓
Agent B
 ↓
Agent B devam eder
```

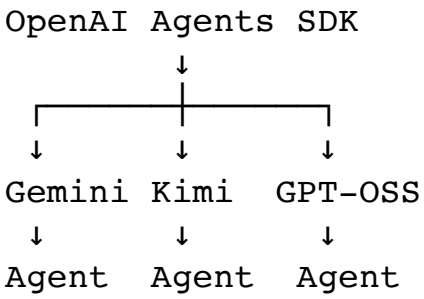
Temel ezber:

Code Orchestration'da akışı geliştirici; LLM Orchestration'da akışın daha büyük kısmını LLM belirler.

11. Multi-Model Agent Sistemleri

OpenAI Agents SDK kullanmak bütün Agent'ların OpenAI modeli kullanması gerektiği anlamına gelmez.

Derste örneğin farklı Agent'larda farklı modeller kullanıldı.



Buradaki önemli ayrım:

Agents SDK = Framework
Gemini/GPT/Kimi vb. = Agent'ın kullandığı LLM

Bunun avantajı farklı görevlerde hız, maliyet ve kalite açısından farklı modeller seçebilmek.

12. Structured Output

LLM normalde serbest metin üretir:

```
"Bence üçüncü mail daha iyi."
```

Fakat Python'un bununla karar vermesi zordur.

Structured Output ile:

```
{
  "selected_email_id": 3,
  "score": 9,
  "approved": true
}
```

gibi düzenli veri alabiliriz.

Tanım:

Structured Output, LLM'in serbest metin yerine önceden belirlenmiş bir veri yapısında çıktı üretmesini sağlar.

Pydantic

İstediğimiz yapıyı Pydantic ile tanımlıyoruz:

```
class EmailSelection(BaseModel):
```

```
    selected_email_id: int
    score: int
    reason: str
    approved: bool
```

Agent'a:

```
output_type=EmailSelection
```

verebiliriz.

Perde arkasındaki temel fikir:

```
Pydantic Model
 ↓
JSON Schema
 ↓
LLM
 ↓
JSON
 ↓
Pydantic
 ↓
Python Object
```

Böylece:

```
selection.approved
selection.score
```

gibi doğrudan kod içerisinde kullanılabilecek değerlere sahip oluruz.

13. Constrained Decoding

Structured Output sırasında modelin belirlenen şemaya uymasına yardımcı olmak için geçersiz token seçimleri kısıtlanabilir.

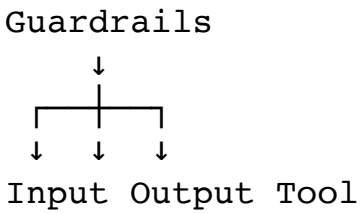
Kısa tanım:

Constrained Decoding, modelin belirlenen çıktı şemasını bozacak tokenları üretmesini kısıtlayan tekniktir.

Ayrıca Structured Output şemalarını gereksiz yere çok karmaşık yapmamak gerektiğini öğrendik.

14. Guardrails

Guardrails, Agent sisteminin girişlerini, çıkışlarını veya tool işlemlerini kontrol eden mekanizmalardır.

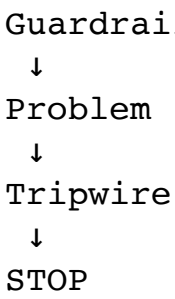


Input Guardrail: Kullanıcı girdisini kontrol eder.

Output Guardrail: Agent'ın final çıktısını kontrol eder.

Tool Guardrail: Tool çağrılarını kontrol eder.

Bir sorun bulunursa **Tripwire** tetiklenip yürütme durdurulabilir.



Ayrıca her guardrail'in framework özelliği olmak zorunda olmadığını gördük.

Bazen basit:

```
if not approved:
    return
```

kontrolü daha anlaşılır ve kontrollü olabilir.

15. Structured Output ile Guardrail Arasındaki Fark

Bu ikisini ayırmak önemli:

```
Structured Output
       ↓
"LLM sonucu hangi formatta verecek?"


Guardrail
       ↓
"Bu sonuç kurallara uygun mu?"
```

Birlikte:

```
Picker Agent
 ↓
Structured Output
 ↓
approved = True
score = 9
 ↓
Python Guardrail
 ↓
Kurallara uygun mu?
 ↓
Evet → Devam
Hayır → STOP
```

16. Sandbox Agent

Sandbox, Agent'a kontrollü bir çalışma ortamı sağlar.

```
Sandbox Agent
 ↓
Dosya oku
 ↓
Kod incele
 ↓
Terminal çalıştır
 ↓
Dosya oluştur/değiştir
```

Manifest benzeri izin mekanizmalarıyla Agent'ın hangi alanlara erişebileceği sınırlandırılabilir.

Yani Agent'a bütün bilgisayarı vermek yerine **izole ve kontrollü bir çalışma alanı** sağlanır.

17. MCP

MCP'nin temel amacı:

Agent'ın harici araçlara, servislere ve dış bilgi kaynaklarına standart bir şekilde erişmesini sağlamaktır.

```
Agent
 ↓
MCP Client
 ↓
MCP Server
 ↓
Tool
 ↓
Dış Sistem
```

Örneğin:

```
Agent
 ↓
MCP
 ↓
Web
 ↓
Güncel bilgi
```

Böylece LLM yalnızca kendi eğitim bilgisinden cevap üretmek zorunda kalmaz.

18. Deep Research Agent Örneği

Derste bütün bu kavramların birleştiği başka bir proje gördük: **Deep Research Agent**.

Akışı:

```
Kullanıcı Sorusu
 ↓
Planner Agent
 ↓
Structured Search Plan
 ↓
```

```
Search Agent
 ↓
Web Search Tool
 ↓
Araştırma sonuçları
 ↓
Writer Agent
 ↓
Kapsamlı Rapor
 ↓
Email Agent
 ↓
Rapor gönderilir
```

Burada özellikle **Code Orchestration** kullanılıyor çünkü akış zaten belli.

Planner → Search → Writer → Email.

LLM'nin hangi Agent'ın çalışacağını yeniden keşfetmesine gerek yok.

Hosted Tools

Search Agent tarafında hazır `WebSearchTool()` gibi Hosted Tool kavramını gördük.

Hosted Tool, sağlayıcının kendi altyapısında çalıştırdığı hazır araçtır.

Örneğin dosyada Web Search, File Search, Code Interpreter, Hosted MCP ve Image Generation gibi örnekler geçiyor.

19. Sonra Kendi Projemizi Tasarladık

Bütün öğrendiklerimizi sadece teoride bırakmadık.

AI Sales Outreach Assistant projesini geliştirdik.

Amaç:

Bir hedef şirketi araştıran, o şirkete özel birden fazla satış maili hazırlayan, en iyi maili seçen, güvenlik kontrolünden geçiren ve gerçek mail gönderen Multi-Agent satış sistemi oluşturmak.

20. Projemizin Agent'ları

Projede temel olarak:

```
Planner Agent

Professional Writer Agent

Creative Writer Agent

Executive / Direct Writer Agent

Picker Agent
```

kullandık.

Burada ayrıca **Guardrail bir Agent değil**, Python kontrol katmanımızdır.

21. Planner Agent

Planner'ın iki görevi var:

```
Kullanıcı isteği
 ↓
Planner Agent
 ↓
Şirketi belirle
+
Araştırma sorgularını oluştur
```

Örneğin:

```
"open ai ya satış maili gönder"
```

gibi doğal/hatalı bir ifadeden:

```
company_name = OpenAI

queries:
- ürünleri
- hedef kitlesi
- güncel gelişmeleri
- iş birliği noktaları
```

oluşturabiliyor.

Başlangıçta şirket adını Regex ile çıkarmaya çalıştık fakat doğal dil karşısında kırılgan olduğu için bunu Planner'a verdik.

22. ResearchPlan — Structured Output

Planner'ın serbest metin vermesini istemedik.

Pydantic ile:

```
ResearchPlan
├── company_name
└── queries[]
```

oluşturduk.

Yani bugün öğrendiğimiz **Structured Output'u gerçek projemizde kullandık.**

23. Sales Manager ve Pipeline

Projemizin ana orkestrasyon dosyası:

```
sales_manager.py
```

Buradaki:

```
run_sales_pipeline()
```

bütün iş akışını yönetiyor.

Pipeline nedir?

Pipeline, bir işlemin başlangıçtan sonuca kadar hangi aşamalardan ve hangi sırayla geçeceğini belirleyen iş akışıdır.

Bizim pipeline:

```
Planner
 ↓
Research
 ↓
Writers
 ↓
```

```
Picker
 ↓
Guardrail
 ↓
Send Email
```

Sales Manager nedir?

Sales Manager bu pipeline'ı yöneten **Python Code Orchestration katmanımızdır.**

Yani:

```
Pipeline
= gidilecek yol

Sales Manager
= o yolu yöneten kod
```

Akışımız zaten belli olduğu için LLM Orchestration yerine Code Orchestration kullandık.

24. MCP ile Gerçek Şirket Araştırması

Planner sorguları oluşturduktan sonra:

```
Planner
 ↓
queries
 ↓
MCP Client
 ↓
MCP Server
 ↓
research_company()
 ↓
DuckDuckGo
 ↓
Web
 ↓
Research Context
```

akışını kurduk.

Böylece Writer'lar yalnızca LLM'in kendi bilgisinden değil, **web araştırmasından gelen context'ten** mail oluşturuyor.

25. Şirket E-mail Adresini Otomatik Bulma

Başlangıçta kullanıcıdan alıcı mailini zorunlu istiyorduk.

Sonra sistemi geliştirdik:

```
Kullanıcı mail yazdı mı?
 ↓
Evet → Manuel adresi kullan

Hayır
 ↓
MCP
 ↓
Şirketin resmi/uygun iletişim mailini araştır
 ↓
Bulundu → Kullan
Bulunamadı → Manuel adres iste
```

Önemli kural:

Sistem bulamadığı e-mail adresini uydurmamalı.

26. Eski E-mailin Yeni Şirkette Kullanılması Problemi

Burada gerçek bir state problemi yakaladık.

İlk işlem:

```
OpenAI
 ↓
MCP → OpenAI mailini buldu
 ↓
UI input'una yazıldı
```

Sonra:

```
Google
 ↓
Input'ta OpenAI maili hâlâ duruyor
 ↓
Sistem bunu manuel mail sandı
```

Bunu düzelttik.

Artık MCP'nin otomatik bulduğu adres **manuel input alanına kalıcı şekilde taşınmıyor**.

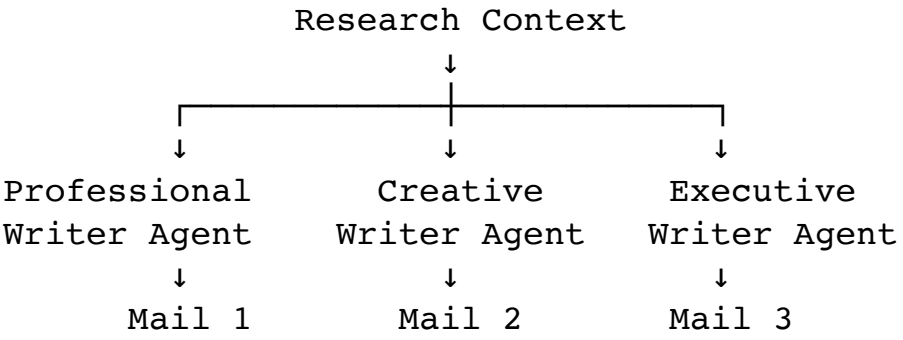
Her yeni şirket için:

```
Google
 ↓
MCP
 ↓
Google adresini yeniden araştır
```

yapılıyor.

27. Üç Writer Agent

Research Context hazırlandıktan sonra:



oluşturuyoruz.

Professional → Kurumsal.

Creative → Yaratıcı/dikkat çekici.

Executive → Kısa, doğrudan, yönetici seviyesinde.

Böylece tek bir LLM çıktısına bağlı kalmıyoruz.

28. AsyncIO'yu Gerçek Projede Kullandık

Teoride öğrendiğimiz `asyncio.gather()` burada gerçek işe dönüştü.

Writer'ları:

```
await asyncio.gather(...)
```

ile birlikte ilerlettik.

Aynı şeyi bağımsız araştırma sorgularında da kullandık.

```
Search 1 ─┐
Search 2 ─┼→ asyncio.gather()
Search 3 ─┤
Search 4 ─┘
```

Yani bugün öğrendiğimiz:

```
async
await
Coroutine
Event Loop
asyncio.gather()
```

kavramlarının gerçek kullanımını projemizde gördük.

29. Picker Agent

Üç mail:

```
Mail 1 ─┐
Mail 2 ─┼→ Picker Agent
Mail 3 ─┘
```

şeklinde Picker'a gidiyor.

Picker en iyi maili seçiyor ve Pydantic `EmailSelection` döndürüyor:

```
EmailSelection
├── selected_email_id
```

```
├── score
├── reason
└── approved
```

Burada yine **Structured Output** kullanıyoruz.

30. Python Guardrail

Picker'dan sonra başka bir Verify Agent eklemek yerine kesin kuralları Python'a bıraktık.

```
Picker
 ↓
Python Guardrail
 ↓
PASS / FAIL
```

Kontroller:

- Picker onayladı mı?
- Score yeterli mi?
- Mail gerçekten var mı?
- Alıcı adresi uygun mu?

Buradaki mühendislik prensibimiz:

Yorumlama gereken işi LLM'e, kesin kuralları Python'a ver.

31. MCP ile Mail Gönderme

Maili gerçekte MCP göndermiyor.

SMTP gönderiyor.

Biz istersek:

```
Application
 ↓
SMTP
 ↓
Gmail
```

yapabilirdik.

Ama:

```
Application
 ↓
MCP Client
 ↓
MCP Server
 ↓
send_email Tool
 ↓
SMTP
 ↓
Gmail
```

yaptık.

Çünkü MCP sayesinde dış servis entegrasyonunu ana Agent uygulamasından ayırıp standart bir tool katmanı haline getiriyoruz.

Kısaca:

SMTP maili gerçekten gönderir; MCP Agent sisteminin bu yeteneğe erişmesini sağlar.

32. Tracer — Kendi Projemizde Observability

Burada önemli bir ayrım yaptık.

Derste Agents SDK'nın `trace()` özelliğini öğrendik.

Kendi projemizde ise ayrıca **custom `Tracer`** geliştirdik.

Her çalıştırmaya:

```
run_id
```

veriyoruz.

Ve:

```
planner
mcp_research
writers_parallel
picker
```

```
send_email
```

gibi pipeline aşamalarını logluyoruz.

Örneğin:

```
planner          → 2.81 sn
mcp_research     → 22.87 sn
writers_parallel → 4.80 sn
picker           → 1.92 sn
send_email       → 2.92 sn
```

Böylece:

Hata nerede çıktı?

Sistem neden yavaşladı?

gibi soruları cevaplayabiliyoruz.

Kısa tanım

Tracer, pipeline çalışırken hangi adımların gerçekleştiğini, ne kadar sürdüğünü ve nerede problem çıktığını takip edip loglayan observability mekanizmamızdır.

33. Streaming'i Projemizde Nasıl Kullandık?

Pipeline'ın tamamının bitmesini bekleyip kullanıcıya tek cevap vermiyoruz.

Her aşamada:

```
yield {...}
```

kullanıyoruz.

Gradio tarafında:

```
async for event in run_sales_pipeline(...):
```

ile bu eventleri geldikçe alıyoruz.

```
Pipeline
 ↓
yield event
```

```
 ↓
Gradio async for
 ↓
UI güncellenir
```

Kullanıcı:

```
Hedef şirket belirleniyor...
↓
Araştırma yapılıyor...
↓
3 mail hazırlanıyor...
↓
Picker çalışıyor...
↓
Guardrail kontrol ediyor...
↓
Mail gönderiliyor...
```

şeklinde süreci canlı görebiliyor.

Buradaki ayrım:

Tracer → geliştirici için arka plan takibi.

Streaming → kullanıcı için UI'da canlı ilerleme.

34. Gradio UI

Son kullanıcı için Gradio arayüzü geliştirdik.

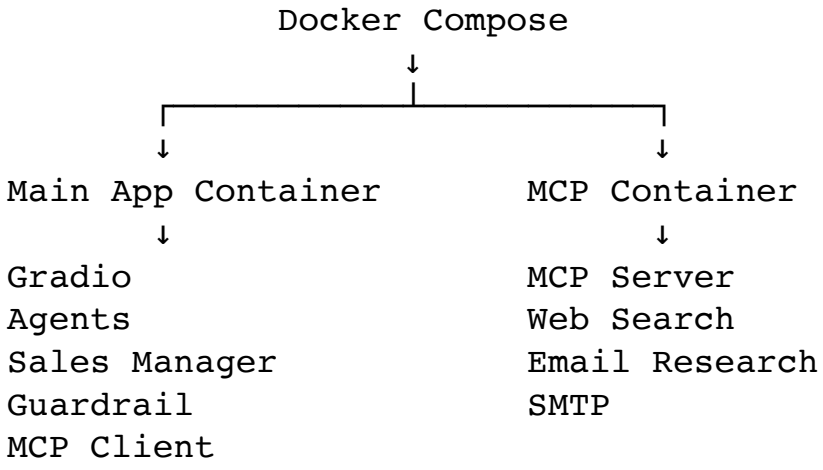
Temel olarak:

```
Satış İsteği
+
Alıcı E-mail
(Opsiyonel / Manuel Override)
 ↓
Satış Mailini Oluştur ve Gönder
 ↓
Gönderilen Mail
 ↓
Analiz
```

Kullanıcı arka plandaki Agent, MCP, SMTP, async gibi ayrıntılarla uğraşmıyor.

35. Docker Compose

Sistemi iki ana servise ayırdık:



Böylece ana Agent uygulaması ile dış entegrasyon/tool katmanı birbirinden ayrılmış oldu.

36. Docker Volume

Loglarımızı yalnızca container'ın içine yazarsak container silindiğinde logları kaybedebiliriz.

Bu nedenle:

```
Container
 ↓
logs/
 ↓
Docker Volume
 ↓
Kalıcı veri
```

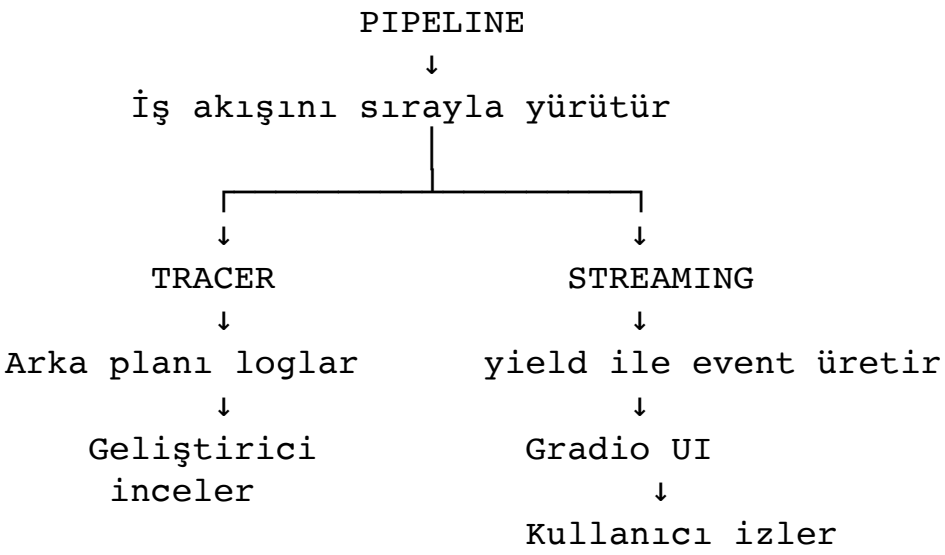
mantığını kullandık.

Sunum cümlen:

“Tracer ve mail loglarını Docker Volume ile kalıcı hale getirdik; böylece container silinse veya yeniden oluşturulsa bile loglarımız kaybolmuyor.”

37. Pipeline + Tracer + Streaming İlişkisi

Bugün en sonunda oturttuğumuz önemli ilişki buydu:



Bunu tek cümlede şöyle anlatabilirsin:

“Arka planda pipeline'ın işleyişini Tracer ile logluyor, streaming mantığıyla da pipeline'daki gelişmeleri Gradio arayüzünde kullanıcıya canlı gösteriyoruz.”

38. OpenAI Agents SDK ile Kendi Yazdıklarımızı Ayırmak

Bu da bugün netleştirdiğimiz önemli noktalardan biri.

```
OpenAI Agents SDK
 ↓
Agent
Runner
Agent Loop
Tools
SDK Trace
SDK Streaming
Memory / Session
MCP entegrasyon yapıları
```

Ama bizim projede ayrıca kendimiz:

```
Sales Pipeline
Code Orchestration
Custom Tracer
yield tabanlı UI progress
Python Guardrail
Gradio UI
MCP Server tool'ları
Docker mimarisi
```

geliştirdik.

Dolayısıyla:

Agents SDK bize Agent'ları oluşturmak ve çalıştırmak için altyapı sağladı; biz bunun üzerine kendi satış pipeline'ımızı ve uygulama mimarimizi kurduk.

39. Projenin Tam Akışı

```
KULLANICI
   ↓
GRADIO UI
   ↓
SALES MANAGER
   ↓
PIPELINE
   ↓
PLANNER AGENT
   ↓
Structured Output
   ↓
ResearchPlan
   ↓
MCP CLIENT
   ↓
MCP SERVER
   ↓
DuckDuckGo / Web Research
   ↓
Research Context
   ↓
asyncio.gather()
   ↓
Professional Writer
Creative Writer
```

```
Executive Writer
   ↓
3 farklı mail
   ↓
PICKER AGENT
   ↓
Structured Output
   ↓
EmailSelection
   ↓
PYTHON GUARDRAIL
   ↓
PASS?
 ↓     ↓
Hayır  Evet
 ↓      ↓
STOP   MCP SERVER
         ↓
     send_email
         ↓
        SMTP
         ↓
       Gmail
         ↓
    GERÇEK E-MAIL
```

Bu sırada:

```
Pipeline
   ↓
Tracer → Arka plan logları

Pipeline
   ↓
yield / Streaming → Gradio canlı ilerleme

Docker
   ↓
Volume → Logların kalıcılığı
```

40. Bugünün En Önemli Kavramları

Bugünkü notların tamamını birkaç ana gruba ayırırsak:

Konu	Ne öğrendik / kullandık?
OpenAI Agents SDK	Agent framework
Agent	Model + Instructions + Tools
Runner	Agent Loop'u çalıştırır
AsyncIO	Bekleme sürelerini verimli kullanır
async / await	Asenkron çalışma
Coroutine	Duraklatılabilir async yapı
Event Loop	Async işleri yönetir
asyncio.gather	Bağımsız async işleri concurrent ilerletir
Multi-Agent	Birden fazla uzman Agent
Orchestration	Agent'ların çalışma düzeni
Code Orchestration	Akışı Python belirler
Agents as Tools	Agent başka Agent'ı tool olarak kullanır
Handoff	Görev başka Agent'a devredilir
Multi-Model	Farklı Agent'larda farklı LLM'ler
Structured Output	LLM'den düzenli veri alma
Pydantic	Veri şemasını tanımlama
Constrained Decoding	Şemaya uymayan tokenları kısıtlama
Guardrail	Güvenlik/kural kontrolü
Tripwire	Sorunda yürütmeyi durdurma
Tools	Agent'ın dış işlem yapması
Memory	Geçmiş context'i koruma
MCP	Dış araç/servis entegrasyonu
Pipeline	Uçtan uca işlem akışı
Tracer	Pipeline'ı gözlemleme/loglama
Streaming	Kullanıcıya canlı ilerleme
Gradio	Kullanıcı arayüzü
SMTP	Gerçek mail gönderimi
Docker Compose	Servisleri containerize etme
Docker Volume	Logları kalıcı tutma

Günün tek büyük resmi

Bugüne aslında **“OpenAI Agents SDK nedir ve AsyncIO neden gerekiyor?”** diye başladık.

Sonra:

```
AsyncIO
 ↓
Agents SDK
 ↓
Agent + Runner
 ↓
Tools + Memory
 ↓
Trace + Streaming
 ↓
Multi-Agent
 ↓
Orchestration
 ↓
Structured Output
 ↓
Guardrails
 ↓
MCP
 ↓
Deep Research örneği
 ↓
Kendi gerçek projemiz
 ↓
AI SALES OUTREACH ASSISTANT
```

noktasına geldik.

Yani bugün yalnızca kavram öğrenmedik; **Agent, Runner, AsyncIO, Multi-Agent, Code Orchestration, Structured Output, Pydantic, Guardrail, MCP, Tracing, Streaming, Docker ve gerçek dış servis entegrasyonlarını tek çalışan mimaride birleştirdik.**